← 返回记忆卡 打印 / 另存为 PDF ☑ 连「理解铺垫」一起印

点「打印 / 另存为 PDF」，在弹出窗口里把「目标」选为「另存为 PDF」即可保存。 每张卡的**结论**单独成块——想自测就把结论那半栏用纸盖住，看着标题先自己背一遍。 默认不印「理解铺垫」（那是理解用的长文，会让页数翻几倍）；需要就勾上面的选项。

计算机网络 · 记忆卡

codebrick.tech · 用户ID 14127 · 2026-10-27 · 共 45 张

计算机网络体系结构 4 张

网络分层与参考模型 高频 对比表

- 三模型对应（自顶向下）：

OSI 七层	TCP/IP 四层	五层模型	PDU
应用层 / 表示层 / 会话层	应用层	应用层	报文
传输层	传输层	传输层	段
网络层	网际层	网络层	数据报/分组
数据链路层 / 物理层	网络接口层	数据链路层 / 物理层	帧 / 比特

- OSI 口诀（自下而上）：「物联网叔会使用」= 物理—链路—网络—传输—会话—表示—应用（联=链、叔=输、使=示）。
- 各层协议归属：应用层 = HTTP/FTP/DNS/SMTP/DHCP；传输层 = TCP/UDP；网络层 = IP/ICMP/IGMP/ARP（按教材）/OSPF（直接封装在 IP 中）；RIP（UDP 520）、BGP（TCP 179）按协议栈位置属应用层，功能上为网络层选路；数据链路层 = 以太网(802.3)/PPP/802.11。
- PDU 串：报文 → 段 → 数据报 → 帧 → 比特。
- 协议三要素：语法 + 语义 + 时序（也常译作同步）。协议（水平，对等层之间）≠ 服务（垂直，下层给上层）≠ 接口/SAP。
- 关系式：n-SDU + n-PCI = n-PDU。
- 设备工作层：中继器/集线器=物理；网桥/二层交换机=数据链路；路由器/三层交换机=网络。

这张卡背什么

体系结构是 CN 第一高频考点（分层考过 11 次），几乎年年出。这张卡承担两件事：**三模型的层次对应**和**各层协议/设备归属**。具体计算（如封装开销、传输效率）交给题，这里只锚结构。

三模型怎么对齐

OSI 七层是理论参考、TCP/IP 四层是因特网实际采用、五层模型是 408 教学默认框架（兼顾 OSI 的清晰与 TCP/IP 的实用）。对齐的关键是两处合并/拆分：

- OSI 的**表示层 + 会话层**功能弱，TCP/IP 把它们**并进应用层**。
- TCP/IP 的**网络接口层**在五层模型里**拆成数据链路层 + 物理层**。

记住这两处，三张表就能互相换算。命题方爱考"OSI 第 5 层（自下而上）是哪层"——自下而上数：物1-链2-网3-传4-会5，第 5 层是会话层，别从上往下数错方向。

各层协议归属（整改吸收点）

这是选择题的送分点，但 ARP 是经典陷阱：

- **应用层**：HTTP、FTP、DNS、SMTP/POP3/IMAP、DHCP；RIP、BGP 按协议栈位置也在这一层（见下）
- **传输层**：TCP、UDP
- **网络层**：IP、ICMP、IGMP、**ARP（按 408 教材归网络层）**、OSPF（直接封装在 IP 数据报中）

- **数据链路层**：以太网(IEEE 802.3)、PPP、IEEE 802.11

ARP 功能上是 IP→MAC 的映射、跨着链路层概念，但 408 教材（谢希仁版）固定把它归在**网络层**，考试按教材答。

路由协议是另一处陷阱：RIP 报文封装在 UDP（端口 520）里，BGP 跑在 TCP（端口 179）上，按协议栈位置都属于应用层，只是功能上为网络层选路；OSPF 直接封装在 IP 数据报中，才在网络层。

协议 / 服务 / 接口别混

- **协议**：对等层之间的规则，**水平**方向；由语法、语义、时序三要素组成（有的教材把 timing 译作“同步”）。
- **服务**：下层给紧邻上层提供的功能，**垂直**方向。
- **接口（SAP）**：同一结点内相邻两层交换信息的逻辑点。

一句话锚点：**协议横、服务纵**。不能把"TCP 协议"说成"TCP 服务"。

三种通信范围跃迁

数据链路层 = **相邻结点**（MAC 地址）；网络层 = **主机到主机**（IP 地址）；传输层 = **端到端 / 进程到进程**（端口号）。这条"节点间→主机间→进程间"的跃迁是反向题（给动作问层次）的判题钥匙。

> 封装/解封装过程、PDU/SDU/PCI 的完整推演、设备隔离冲突域/广播域的细节，点「展开博客精讲」与可视化看动态演示，记忆卡只锚结构骨架。

网络性能指标 中频 易错辨析

- 死亡陷阱（三者别混）：
  - 带宽 = 链路能传的**最高**发送速率（理论上限），单位 bit/s。
  - 吞吐量 = 单位时间**实际**通过的数据量；多段串联由**瓶颈链路**（带宽最小那段）决定。
  - 时延带宽积 = 传播时延 × 带宽 = 链路"容量"（某时刻正在链路上传播的比特数）。用的是**传播时延**，不是发送时延。
- 四种时延：总时延 = 发送时延 + 传播时延 + 排队时延 + 处理时延。
  - 发送时延 = 数据长度 / 发送速率（与数据量、速率有关，与距离无关）。
  - 传播时延 = 信道长度 / 传播速率（与距离有关，与数据量无关）。
- RTT ≈ 2 × 传播时延（往返时间）。
- 单位换算辨析：网络速率 b/s 的 k/M/G = **10^3/10^6/10^9**；存储容量 B 的 K/M/G = **2^{10}/2^{20}/2^{30}**（1 KB = 1024 B）。两套进制不能混用。

这张卡背什么

性能指标是本章 high 频考点，但**具体数值计算交给题和时延计算器**，这张卡只锚"哪些量容易混、各自定性是什么"。把三组辨析钉死就能挡住大部分送分陷阱：带宽 vs 吞吐量 vs 时延带宽积、发送时延 vs 传播时延、网络进制 vs 存储进制。

死亡陷阱：带宽 / 吞吐量 / 时延带宽积

这三个量都跟"带宽"沾边，最容易糊成一团：

- **带宽**是"水管最粗能流多快"——理论上限。
- **吞吐量**是"此刻实际流了多少"——受拥塞、瓶颈链路、收发能力压制，多段串联只看最窄那段。
- **时延带宽积**是"这根管子里此刻装着多少水"——是个**容量**概念，等于传播时延 × 带宽。

最常见错法是算时延带宽积时误用发送时延：记牢公式里那个时延是**传播时延**。

发送时延 vs 传播时延

这是计算题的命门，两者完全独立：发送时延只跟**数据量 / 发送速率**有关、跟距离无关；传播时延只跟**距离 / 传播速率**有关、跟数据量无关。直觉记法——发送时延是"把水倒进管口"的时间，传播时延是"水从管子这头流到那头"的时间。水多则倒得久，管长则流得久。

两套进制不能混（整改吸收点）

- **网络速率**（bit/s）里的 k、M、G 是 **10^3**、**10^6**、**10^9**，10 进制。
- **存储容量**（字节 B）里的 K、M、G 是 **2^{10}**、**2^{20}**、**2^{30}**，即 1 KB = 1024 B。

做题先看单位是"比特每秒"还是"字节"，进制选错整道计算全错。这一条是命题方反复埋的坑。

> 各时延的代入计算、时延带宽积/总时延的数值演算交给「时延计算器」和题目，记忆卡只锚定性结论与公式名。

网络分类 冷门 对比表

- 按覆盖范围（由大到小）：WAN 广域网 > MAN 城域网 > LAN 局域网 > PAN 个人区域网。

类型	范围量级	典型技术
WAN 广域网	几十~几千公里	PPP、HDLC
MAN 城域网	一个城市	以太网技术为主
LAN 局域网	几十米~几公里	以太网、Wi-Fi
PAN 个人区域网	几米	蓝牙

- 按传输技术：广播式网络（共享一条信道，靠目的地址决定收不收，LAN 多用）vs 点对点网络（每条线连一对结点，无直连则存储转发）。
- 局域网常用广播信道，广域网多用点对点 + 分组交换。

这张卡背什么

网络分类是选择题级的轻考点，最常考两条线：**按覆盖范围**的四档大小排序，和**按传输技术**的广播式 vs 点对点。其它分类（按使用者公用/专用、按拓扑、按交换方式）了解即可，不必死记。

范围排序怎么记不混

四个量级是一根连续的尺子：人身边几米（PAN，蓝牙）→ 一栋楼/校园几公里（LAN，以太网+Wi-Fi）→ 一座城（MAN）→ 跨省跨国（WAN）。命题方常把"典型技术"和"范围"错配来设错项（例如把 PPP 安到局域网上），记住 **PPP/HDLC 是广域网链路技术、以太网/Wi-Fi 是局域网技术**就能排除。

广播式 vs 点对点：判断关键

这是按"传输技术"分类的核心辨析：

- **广播式网络**：所有主机共享一个公共信道，任一主机发的帧大家都能收到，靠检查**目的地址**决定是否接收——局域网普遍采用。
- **点对点网络**：每条物理线路只连一对结点，没有直连就得经中间结点**存储转发**——广域网典型形态。

一句话锚点：**共享信道靠地址过滤 = 广播式；独占线路靠转发接力 = 点对点**。

> 拓扑结构、按使用者/交换方式等更细的分法在源博客有完整表格，点「展开博客精讲」查阅，记忆卡只锚最常考的两条线。

网络概念组成与功能 冷门 定义锚点

- 一句话定义：把分散、自治的计算机系统用通信线路连起来，实现资源共享 + 信息传递。
- 组成三视角：① 组成部分 = 硬件 + 软件 + 协议（协议是核心）；② 工作方式 = 边缘部分（用户主机，C/S 与 P2P）+ 核心部分（网络与路由器，做分组交换）；③ 功能组成 = 通信子网 + 资源子网。
- 两大基本功能：数据通信（连通）+ 资源共享。
- internet（互连网，通用名词）≠ Internet（因特网，特指全球 TCP/IP 那张网）。

这张卡背什么

`network-overview` 是整章最轻的考点，选择题里只会拿来做**概念辨析**，不会单独出计算。背三件事就够：定义、三视角组成、两大功能。剩下的交换方式细节和性能指标，分别由 `network-classification` 、 `network-performance` 两张卡承接，这里不展开。

为什么"组成"要分三个视角

同一张网，换个角度问就是不同的考点，命题方常拿"组成"挖坑：

- **组成部分**（硬件/软件/协议）回答"网络由什么搭起来"——把协议也算进"组成"是易错点，协议不是配件而是核心。
- **工作方式**（边缘/核心）回答"谁在用、谁在转"——边缘是用户直接使用的主机，核心是为边缘提供连通的路由器群。
- **功能组成**（通信子网/资源子网）回答"哪部分负责传、哪部分负责给"。

三个视角不是三种网络，是看同一张网的三副眼镜。选择题给你一个名词（如"资源子网"），要你判断它属于哪个视角，记混就丢分。

一个高频陷阱：internet 与 Internet

408 选过这个大小写辨析。**internet** 是通用名词，泛指任意多网互连、不限定协议；**Internet** 是专有名词，特指当前全球最大、用 TCP/IP 协议族的那张具体网络。判断口诀：小写讲"互连这件事"，大写讲"因特网那张网"。

> 交换方式（电路/报文/分组）、网络分类（LAN/MAN/WAN）虽与本节同源，但属其它 KP，详见对应卡片，点「展开博客精讲」看完整对比。

物理层 5 张

通信基础 高频 易错辨析

- 码元 ≠ 比特：1 个 M 进制码元 = $\log_2 M$ 比特；只有 2 进制时码元数 = 比特数
- 波特率（码元/秒，Baud）≠ 比特率（bit/s）：比特率 = 波特率 × $\log_2 V$
- 单工（永远一个方向）/ 半双工（双向但不能同时）/ 全双工（双向且同时）
- 链路（物理线路）≠ 信道（逻辑通路）：一条链路可复用出多个信道
- 基带（不调制直接传，近距离）/ 频带（调制搬到高频，远距离）
- 调制三法：ASK 调幅 / FSK 调频 / PSK 调相，每码元各 1 bit

物理层最冷的章，但「概念辨析」年年出选择题。**这张卡只记概念区分，速率换算的算题交给练习**——记住公式名和方向即可，别在卡里背中间数值。

码元 vs 比特：码元是信号里携带信息的最小单位（一个波形变化），比特是信息量单位。一个码元有 V 种状态时携带 $\log_2 V$ 比特，所以只有二进制（V=2）时两者数量才相等。这个区分是后面奈奎斯特定理的前提。

波特率 vs 比特率：波特率数的是「每秒几个码元」，比特率数的是「每秒几个比特」。常见坑：4 相位调制时 V=4，比特率 = 波特率 × $\log_2 4$ = 波特率 × 2，不要把「4 相位」的 4 直接拿去乘除。

单工 / 半双工 / 全双工：易错点在半双工——它是「双方都能发，但不能同时发」（对讲机），不是「只有一个方向能发」。永远只有一个方向的才是单工（广播、电视）。

基带 ≠ 数字传输：基带传输强调「不调制、频带从基频开始」，传的信号仍可以是编过码的（如曼彻斯特）。别把「基带」等同于「未编码的数字信号」。

> 点「展开博客精讲」看信道四种数据/信号转换关系、PCM 三步与电话 64 kbit/s 的来历。

编码与调制 低频 对比表

编码	自同步	规则	典型应用
NRZ	无	高=1 低=0，整码元电平不变	很少单用
曼彻斯特	有	看码元「中点」跳变方向定 0/1	以太网
差分曼彻斯特	有	看码元「起点」有无跳变定 0/1，抗干扰更强	令牌环

- 谢希仁教材约定：曼彻斯特 1=高→低、0=低→高；差分曼彻斯特 起点有跳变=0、无跳变=1
- 曼彻斯特/差分曼彻斯特：每码元中间必跳变（自带时钟）→ 波特率 = 2 × 比特率
- 调制三法：ASK 调幅 / FSK 调频 / PSK 调相

典型考法：给波形判编码，或给编码画波形。**波形画法交给可视化**，这张卡记「哪种自同步、哪种看哪里、谁用在哪」。

怎么区分四种：先看码元「中点」是否总有跳变——有则是曼彻斯特或差分曼彻斯特，没有则是 NRZ/NRZI。再细分：曼彻斯特看中点跳变「方向」直接定 0/1；差分曼彻斯特看码元「起点」有无跳变定 0/1（中点跳变只用于同步、与数据无关），所以即使信号极性整体反转也能正确解码，抗干扰更强。

曼彻斯特为什么「波特率 = 2 × 比特率」：每个比特周期内必有一次中间跳变，信号变化速率是数据速率的两倍。记这句定性结论即可，对应的带宽换算（如 10 Mbps 以太网）属算题。

调制 vs 编码：编码是「数字数据→数字信号」（NRZ/曼彻斯特）；调制是「数字数据→模拟信号」，三种基本法 ASK 改幅度、FSK 改频率、PSK 改相位，名字首字母对应改的物理量。

> 易错提醒：曼彻斯特 1/0 的跳变方向、差分曼彻斯特「有跳变」对应 0 还是 1，不同教材定义相反；408 按谢希仁教材，做题先确认题目约定。

电路/报文/分组交换 低频 对比表

对比项	电路交换	报文交换	分组交换
建立连接	需要	不需要	不需要
独占线路	是	否	否
存储转发	否	是（整报文）	是（分组）
转发时延	无	高	较低
线路利用率	低	高	非常高
差错控制	不支持	支持	支持

- 口诀：电路看「独占」、报文看「整存整转」、分组看「分片转发」
- 数据报（不建连·独立选路·可能乱序·高层保可靠） vs 虚电路（建逻辑连接·同路·按序·网络层保可靠）
- 因特网用「数据报方式的分组交换」

---

408 选择题常考三种交换方式的对比与各自特点，这张卡记对比维度和两组辨析。

为什么分组交换最优（针对突发数据）：在报文交换基础上把报文切成小分组，缓存需求小、可流水线转发（前一分组在下一跳转发时，后一分组同时发到当前节点），所以时延比报文交换低；某分组出错只重传该分组，不必重传整报文；统计时分复用使线路利用率最高。代价是每个分组都要带头部控制信息、可能乱序需重组。

但分组交换不是永远更好：通信双方有大量持续数据（如视频通话）时，电路交换的专用通路反而效率更高。分组交换的优势场景是「突发性通信」。

虚电路 ≠ 电路交换：虚电路也要「建立连接」，但建的是逻辑通路，链路并不被独占；电路交换建的是物理层面的独占通路。这是最常混的一对。

> 点「展开博客精讲」看三种方式的完整优缺点与适用场景、数据报/虚电路逐项对比。

物理层设备 冷门 定义锚点

- 中继器：物理层、2 端口；功能 = 信号「再生」（放大 + 整形），不是简单放大
- 集线器 hub = 多端口中继器；物理层、半双工、共享带宽（n 台平分总带宽）
- hub 转发 = 广播到除输入外所有端口；物理拓扑星形、逻辑拓扑总线形
- 锚点：hub「不分割冲突域」——所有端口同属一个冲突域、同属一个广播域（对比交换机）

考频不高的送分题，核心就两点：中继器是信号再生（非简单放大）、集线器不能分割冲突域。

再生 ≠ 放大：放大器对模拟信号简单放大，会把噪声一起放大；中继器对数字信号「再生」——重新生成干净的 0/1 波形（放大 + 整形），不累积噪声。这是中继器区别于放大器的本质。

集线器的定位（对比锚点）：hub 本质是多端口中继器，工作在物理层、半双工、共享带宽（如 10 Mb/s 的 hub 接 8 台，每台平均 1.25 Mb/s）。关键结论是它「不分割冲突域」——所有端口在同一冲突域、同一广播域。往上对比：交换机（网桥）能隔离冲突域，路由器能隔离广播域。这条「设备 × 冲突域 × 广播域」的对比是 CN 选择题高命中点。

> 点「展开博客精讲」看 5-4-3 规则（5 段电缆、4 中继器、3 段可连主机）、集线器连不同速率网段整体降速等细节。

传输介质 冷门 对比表

- 导向型：双绞线（UTP 非屏蔽 / STP 屏蔽）、同轴电缆、光纤
- 非导向型：无线电波（全向）、微波（直线·需中继）、红外/激光（视距·不穿墙）

光纤	纤芯直径	光源	距离
多模	较粗	LED	短距离
单模	极细（近波长）	激光器	长距离

- 光纤靠「全反射」传光；双绞线绞合是为「减少电磁干扰」
- 物理层接口四特性：机械（形状引脚）/ 电气（电压距离）/ 功能（电平含义）/ 过程（事件时序）

选择题考频较高。典型考法：给场景选介质、考光纤全反射、双绞线绞合目的、四特性区分。

相对排序记忆：抗干扰与传输距离上，光纤 > 同轴 > 双绞线；其中单模光纤距离最远（数十千米无中继）、多模较短。无线四种里只有无线电波是全向传播，微波/红外/激光都需视距、有强指向性。

两个高频坑：① 传输介质本身「不属于物理层」（在物理层之下，俗称第 0 层），它只承载信号；物理层靠四特性赋予信号语义。② 物理地址（MAC）虽带「物理」二字，但属数据链路层，不在物理层范畴（2018 真题考过）。

电气特性 vs 功能特性（最易混的一对）：电气特性管「多少伏、传多远」（电压范围、距离限制），功能特性管「这个电平什么意思」（高电平 = 二进制 1）。典型迷惑描述「+10~+15V、线长限 15m」说的是电压范围和距离限制，属电气特性，不是功能特性。

> 点「展开博客精讲」看双绞线/同轴/光纤逐项参数、四种无线介质特点、四特性完整对照。

数据链路层 14 张

随机访问 MAC 高频 对比表

- 本章第一高频（14 次）。随机访问家族：
  - ALOHA：纯 ALOHA（想发就发）/ 分隙 ALOHA（只在时隙起点发）
  - CSMA：1-坚持 / 非坚持 / p-坚持（侦听到忙后的不同退让策略）
- CSMA/CD vs CSMA/CA 核心辨析：

维度	CSMA/CD	CSMA/CA
用在	有线以太网（802.3）	无线局域网（802.11）
冲突处理	边发边听，**检测**到冲突就停发	先听后发，发前预约**避免**冲突
能否检测冲突	能	不能（无线信号衰减 + 隐蔽站）
链路层 ACK	无（靠上层）	有（必须，靠 ACK 确认送达）
RTS/CTS	不用	可选，解决隐蔽站

- CSMA/CD 口诀：先听后发、边发边听、冲突停发、随机重发；争用期 = 2τ
- 令牌环对照（轮询访问，非随机）：持令牌者才能发，无冲突、有最大等待上限（确定性时延）

怎么记不混

- **CD 用于有线、CA 用于无线**——选择题最爱的干扰项是「以太网用 CSMA/CA」。钉死：以太网 = CSMA/CD，Wi-Fi = CSMA/CA。
- **为什么无线不能用 CD**：① 无线发射信号远强于接收信号，自己的发射会淹没别人微弱信号，**根本检测不到冲突**；② 隐蔽站问题——两站互相听不到，侦听机制失效。所以无线只能「避免」不能「检测」，于是必须有链路层 ACK 来确认是否送达。
- **CSMA/CA 靠 RTS/CTS + NAV 解决隐蔽站**：发送前用 RTS/CTS 短帧预约，周围站从 RTS/CTS 的持续时间字段设置 NAV（虚拟载波侦听）计时器，倒计时期间不发。
- **令牌环（轮询访问）和随机访问的本质差异**：令牌环「有序、无冲突、有确定时延上限」，随机访问「争用、可能冲突」。

减法红线

CSMA/CD 的二进制指数退避**退避时间计算**（第 k 次冲突从 {0…2^min(k,10)−1} 取 r，等 r×2τ）属算题，**不背退避时间数值**；争用期 2τ 推最小帧长的乘法演算也交给练习题。卡片只锚「CD 检测/CA 避免、CD 有线/CA 无线、CA 必有 ACK + RTS/CTS、争用期 = 2τ」这些定性结论。

> 展开博客精讲：CSMA/CD 协议（ `/cn-blog/posts/datalink/csma-cd.html` ）、CSMA/CA 协议（ `/cn-blog/posts/datalink/csma-ca.html` ）

以太网 / IEEE802.3 中频 硬结论数字

以太网 = **IEEE 802.3**；半双工用 **CSMA/CD**；曼彻斯特编码；无连接、不可靠（不确认不重传）。

MAC 地址（死记）：

- 长度 **48 位 = 6 字节**，前 24 位 OUI（厂商标识）+ 后 24 位设备号。
- **全 1（FF:FF:FF:FF:FF:FF）= 广播地址**；第一字节最低位 = 1 组播、= 0 单播。

以太网帧字段长度（死记）：

字段	长度
目的 MAC / 源 MAC	各 6 B
类型/长度	2 B
数据	**46 ~ 1500 B**（即 MTU=1500）
FCS（CRC-32）	4 B

- **整帧 64 ~ 1518 B**；数据 **46 ~ 1500 B**；**MTU = 1500 B**。
- **最小帧 64 B**：6+6+2+46+4=64；由来是 **64 B = 2τ（争用期）内能检测到冲突**，即发送时间 ≥ 争用期，发送方发完前才来得及发现冲突。不足 46 B 的数据要 **填充（Padding）** 到 46 B。
- 前导码 7 B + SFD 1 B（共 8 B）属物理层同步信号，**不计入帧长**。

怎么记不混

- **三组长度区分清楚**：「**整帧 64~1518**」「**数据 46~1500**」「**MTU=1500**」。MTU 指的是数据字段最大值（1500），不是整帧最大值（1518）。最小数据字段是 **46 B**，不是 64 B——64 B 是含帧头和 FCS 的整帧最小值。
- **最小帧 64 B 的「为什么」是题眼**：以太网要求发送一帧的时间 ≥ 争用期 **2τ**，这样发送方在发完之前一定能检测到冲突。10 Mbps 以太网 2τ=51.2 μs，乘以速率正好 512 bit = 64 B。所以「**最小帧长 = 2τ × 数据率**」，记住这是为了「边发边听」时能赶在发完前发现冲突。反过来：收到小于 64 B 的帧直接当作冲突碎片丢弃。
- **MAC 全 F 是广播**：FF:FF:FF:FF:FF:FF 是二层广播，同一广播域内全收，但**不跨路由器**。
- **前导码不计入帧长**：算帧长从目的 MAC 起、到 FCS 止；前导码 7 B + SFD 1 B 是物理层的事。

  退避时间计算、给定速率算最小帧长等属计算题，交给练习和可视化；卡里只锚长度结论和「最小帧 = 2τ × 速率」这个定性公式。

流量控制与可靠传输 中频 定义锚点

- 流量控制 = 控制发送方速率，防止淹没接收方（链路层用固定大小滑动窗口）
- 可靠传输 = 无差错、按序交付（靠 ARQ：检错 + 确认 + 超时重传）
- 链路层流控 vs 传输层流控：

维度	链路层	传输层（TCP）
控制范围	相邻两节点之间（一跳）	端到端
窗口大小	通常固定	动态调整（取 rwnd、cwnd 较小者）

- 链路层可靠 vs 传输层可靠：每跳正确交付 ≠ 端到端正确交付（路由器内部仍可能丢包）

怎么记不混

这是「滑动窗口三协议（停等 / GBN / SR）」的总纲卡，本身只锚两个定义 + 两组层次区别，具体协议机制看各自的卡。

- **流量控制 vs 可靠传输是两件事**：流控管「发太快把接收方缓冲撑爆」，可靠传输管「丢了 / 错了 / 乱了怎么补」。滑动窗口同时承担这两个职责。
- **为什么端到端可靠仍必要**：即使每一跳链路层都做到了无差错交付，路由器在转发时缓冲区溢出仍会丢包，这部分不归任何一段链路层管，所以传输层还得再做一层端到端可靠。
- **两层流控的区别落在「窗口是否动态」**：链路层窗口大小通常固定；TCP 窗口受接收窗口 rwnd 和拥塞窗口 cwnd 共同约束，取两者最小值，动态变化。

  展开博客精讲：三种 ARQ 协议对比（ `/cn-blog/posts/datalink/sliding-window-compare.html` ）

后退 N 帧（GBN） 中频 硬结论数字

- 发送窗口 $1 \le W_T \le 2^n - 1$（n = 序号位数）
- 接收窗口 $W_R = 1$
- 确认方式：累积确认（ACK n 表示 n 及之前全部收到）
- 出错处理：回退，重传该出错帧 + 其后全部已发未确认帧
- 接收方只按序收，乱序帧一律丢弃（无缓存）

怎么记不混

GBN 的窗口结论是 408 每年挖的坑，**和 SR 搞反直接丢分**。记忆钩子：

- **GBN 接收窗口 = 1**（只认下一个按序的帧），所以由统一约束 $W_T + W_R \le 2^n$ 推出 **发送窗口 ≤ $2^n - 1$**。「接收窗 1、发送窗几乎用满序号空间」是 GBN 的形态。
- 对照记 SR：SR 收发窗口都要占序号空间、各分一半，故各 ≤ 2^{n-1}。**GBN 是 2^n-1、SR 是 2^{n-1}**，背口诀「GBN 减一、SR 减半」。
- **累积确认**是 GBN 的省事处：丢一个 ACK 没关系，后一个 ACK 能把前面的也确认上。代价是出错时要「回退 N 帧」把后续无辜帧也重传一遍。

减法红线

只锚「窗口上界结论 + 累积确认 + 回退重传」。 $W_T + W_R \le 2^n$ 为什么成立的序号重叠推导、给定丢帧场景算重传几帧——交给练习题，卡片不背推导和中间值。

> 展开博客精讲：后退 N 帧协议 GBN（ `/cn-blog/posts/datalink/gbn.html` ）

选择重传（SR） 中频 硬结论数字

- 发送窗口 W_T = 接收窗口 W_R ≤ 2^{n-1}（n = 序号位数）
- 确认方式：逐个确认（每帧单独确认）
- 出错处理：只重传出错的那一帧
- 接收方有缓存，可乱序接收、暂存后按序上交

怎么记不混

SR 窗口结论与 GBN 是配对辨析点，**搞反直接丢分**。记忆钩子：

- **SR 收、发窗口都要占用序号空间**（接收方要缓存乱序帧、发送方要保留未确认帧），两边都占就得**平分序号空间**，故 W_T = W_R ≤ 2^{n-1}（序号空间一半）。这就是「SR 减半」的来由。
- 对照 GBN：GBN **接收窗 = 1** 不占空间，发送窗几乎能用满，故 ≤ $2^n - 1$。一句话区分：**GBN 减一、SR 减半**。
- SR 比 GBN 省带宽（只重传出错帧），代价是实现复杂（接收方要缓存 + 逐个确认）。

减法红线

只锚「窗口上界 W_T = W_R ≤ 2^{n-1} + 逐个确认 + 只重传出错帧 + 接收方有缓存」。为什么是 2^{n-1} 的序号重叠歧义推导、给定 n 求具体窗口数值——交给练习题，卡片不背中间演算。

展开博客精讲：选择重传协议 SR（ `/cn-blog/posts/datalink/sr.html` ）

以太网交换机 中频 对比表

以太网交换机工作在**数据链路层**，靠 **MAC 地址表自学习**转发。

自学习 + 转发（看哪个地址）：

- **学习**：看帧的**源 MAC + 入端口**，写入/刷新 MAC 地址表（表项有老化时间，默认 300s）。
- **转发**：看帧的**目的 MAC**，分三种行为 —— **转发**（表里有、且不是入端口）/ **过滤**（表里有、就是入端口 → 丢弃）/ **泛洪**（表里没有，或目的是广播 → 发往除入端口外所有端口）。
- 一句话：**学源、转目的；广播帧一定泛洪。**

★ 设备 × 冲突域 × 广播域 总对比表（CN 选择题命中率最高，钉死）：

设备	工作层	冲突域	广播域
中继器 / 集线器 Hub（n 口）	物理层	**1 个**（全端口共享）	**1 个**
网桥 / 交换机（n 口）	数据链路层	**n 个**（每端口独立）	**1 个**
路由器（n 口）	网络层	**n 个**	**n 个**（每接口独立）

- 记忆链：**Hub 啥都不隔（1 冲突 1 广播）→ 交换机隔冲突不隔广播（n 冲突 1 广播）→ 路由器全隔（n 冲突 n 广播）**。
- VLAN 可在交换机上再隔广播域（见 vlan 卡）。

怎么记不混

- **学习看源、转发看目的**：自学习阶段记的是「这帧从哪来」→ **源 MAC + 入端口**；只有转发阶段才看**目的 MAC** 去查表。两个阶段看不同地址，别记反。
- **泛洪不含入端口**：泛洪是发往「**除入端口外**的所有端口」，否则帧会回到发送者。两种情况触发泛洪：目的 MAC 查不到、或目的就是广播地址（FF:FF:FF:FF:FF:FF 永远泛洪，哪怕表很全）。
- **设备对比表是 CN 选择题第一命中点**，三句话钉死：
  - **n 口集线器 = 1 个冲突域 + 1 个广播域**（多端口中继器，啥都不隔离）。
  - **n 口交换机 = n 个冲突域 + 1 个广播域**（每端口独立冲突域，但广播帧泛洪到全端口，不隔广播域）。
  - **路由器 = n 个冲突域 + n 个广播域**（三层设备，每个接口既隔冲突又隔广播）。
- 顺口溜：层次越高隔得越多——**物理层 Hub 全不隔、链路层交换机隔一半、网络层路由器全隔**。要在交换机上隔广播域，得靠 **VLAN**。

  给定帧序列分析 MAC 地址表逐步变化、STP 生成树选举过程属过程/机制题，交给练习和「交换机自学习可视化」；卡里只锚自学习规则和设备对比结论。

无线局域网 中频 对比表

无线局域网 = **IEEE 802.11（WiFi）**，所有变种共用同一 MAC 层 **CSMA/CA**。

两种组网模式：

- **基础设施模式**：有 **AP**，站间通信也要经 AP 转发；一个 AP + 其 STA = **BSS**，多 BSS 互连 = **ESS**。
- **自组网 Ad Hoc**：无 AP，节点对等直连。

CSMA/CA vs CSMA/CD（题眼对比）：

维度	CSMA/CD（有线 802.3）	CSMA/CA（无线 802.11）
处理冲突	**冲突检测**（边发边听，撞了停）	**冲突避免**（先听后发、提前预约）
能否边发边听	能	**不能**（自身发射淹没接收信号）
逐帧 ACK	不需要（直接检测冲突）	**必须**（靠 ACK 才知是否送达）
隐蔽站问题	不存在	存在 → 用 **RTS/CTS + NAV** 解决

- **RTS/CTS**（可选，长帧才启用）：发送前用 RTS 预约信道、AP 回 CTS，听到 RTS/CTS 的第三方把占用时长写入 **NAV（虚拟忙倒计时）**，期间不抢信道 → 解决**隐蔽站**。
- 帧间间隔：**SIFS < PIFS < DIFS**（间隔越短优先级越高，CTS/ACK 用 SIFS、普通数据/RTS 等 DIFS）。

怎么记不混

- **为什么无线用 CA 不用 CD（最高频）**：无线是半双工，站点发送时自己的发射信号强度远盖过远端回来的信号，**根本没法「边发边听」检测冲突**；而且信号随距离平方快速衰减、还有隐蔽站。所以放弃「检测冲突」改用「**避免冲突**」——提前预约 + 退避。
- **CSMA/CA 必须有 ACK，CSMA/CD 不需要**：CD 边发边听能直接判断冲突；CA 发不出去也听不到，只能靠接收方显式回 ACK 才知道帧到没到。注意这是 **MAC 层 ACK**（逐帧、链路层），和 TCP 的端到端 ACK 完全不同层。
- **隐蔽站 vs 暴露站**：隐蔽站 =「**该停的没停**」（两站互相听不到，同时向 AP 发会撞）；暴露站 =「**不该停的停了**」（听到邻站在发就误以为不能发，其实方向不冲突）。RTS/CTS 解决的是隐蔽站。
- **NAV 是给「旁观者」用的**：发送方自己知道时长不用 NAV；NAV 是听到 RTS/CTS 的第三方设的「虚拟忙」倒计时。

  DIFS/SIFS/帧长/带宽给定算总发送时间或 NAV 值、802.11 四地址字段的具体解读属计算/细节题，交给练习；卡里只锚 CA/CD 对比、RTS/CTS+NAV 作用、IFS 大小关系。

停止-等待协议 低频 步骤口诀

- 一次只发一帧，停下等 ACK，收到才发下一帧
- 发送窗口 = 1，接收窗口 = 1
- 帧编号只需 1 位（0/1 交替）；序号空间 = $2^1 = 2$
- 超时重传（ARQ）：发一帧就启超时计时器，超时未收 ACK 即重传
- 接收方靠帧编号识别重复帧 → 丢弃数据但补发 ACK

怎么记不混

停等是最朴素的 ARQ，是 GBN / SR 的对比基础。三种异常情况记清楚怎么处理：

- **数据帧丢失/出错**：接收方收不到或检错丢弃，发送方超时后重传。
- **ACK 丢失**：接收方其实已收到，但 ACK 丢了；发送方超时重传同一帧，接收方靠**帧编号**认出是重复帧，丢弃数据、重发 ACK。
- **为什么要编号**：没有编号，接收方无法判断收到的是新帧还是重传帧——这就是 0/1 一位编号存在的理由（任一时刻只有一个未确认帧，只需区分「当前」和「上一」）。

减法红线

信道利用率公式 `U = 1/(1+2a)`（$a = \tau/T_f$）及其代入计算**不背**，交给练习题 + 可视化。卡片只记「窗口都是 1、编号 1 位、超时重传、靠编号识别重复帧」这几条定性结论。

> 展开博客精讲：停止-等待协议（ `/cn-blog/posts/datalink/stop-and-wait.html` ）

VLAN 低频 定义锚点

- VLAN（虚拟局域网）= 在二层交换机上**逻辑划分多个独立广播域**，不需额外路由器即可隔离广播。
- **隔离的是广播域，不是冲突域**（冲突域交换机本身每端口已隔离）。
- 标准 **IEEE 802.1Q**：在以太帧「源 MAC 与类型字段之间」插入 **4 字节 VLAN Tag** = TPID 2 B（固定 0x8100）+ TCI 2 B；TCI 里 **VID 占 12 位**（最多 2^{12}=4096，可用 1~4094）。
- 带标签帧最大 **1522 B**（普通 1518 + 4 B 标签）。
- 端口类型：**Access**（连终端，属单一 VLAN，进打标签 / 出去标签）、**Trunk**（连交换机，帧始终带标签、靠 VID 区分多 VLAN）。
- **跨 VLAN 通信必须经三层设备**（路由器单臂路由 / 三层交换机）；同一 VLAN 跨交换机可经 Trunk 二层互通。

怎么记不混

VLAN 中低频，锚住「隔离广播域 + 802.1Q 4 字节标签 + 跨 VLAN 需三层」三件事即可。

- **VLAN 隔离广播域、不是冲突域**：这是最常见的坑。交换机本来就每端口一个独立冲突域，VLAN 是在此之上**进一步把广播域切开**——同 VLAN 才收彼此的广播帧，不同 VLAN 的广播互不打扰。
- **802.1Q 标签是 4 字节，不是 2 字节**：完整标签 = TPID(2B, 0x8100) + TCI(2B)，别只记住 VID 的 12 位就说「2 字节」。VID 12 位 → 最多 4096 个 VLAN（保留两头，实际 1~4094）。
- **跨 VLAN 一定要三层设备**：纯二层交换机做不到 VLAN 间通信，必须靠路由器（单臂路由）或三层交换机。而「同一 VLAN、不同物理交换机」只要有 Trunk 链路就能二层互通。

  802.1Q 各子字段位宽细节、单臂路由配置等看「展开博客精讲」（ `/cn-blog/posts/datalink/vlan.html` ）。

数据链路层功能与组帧 冷门 对比表

- 三个基本问题（谢希仁）：封装成帧 + 透明传输 + 差错检测（流量控制与可靠传输大纲另列条目，不在这三个里）
- 四种组帧定界：

方法	定界方式	透明传输靠
字符计数法	帧头计数字段标明总字节数	天然透明（计数出错则全部错位、不能恢复）
字符填充法	SOH/EOT 标志	ESC 转义（ESC 本身也要转义）
零比特填充法	01111110（Flag）标志	连续 5 个 1 后插 0
违规编码法	物理层违规信号	天然透明（依赖特定编码）

- 透明传输 = 数据中任何比特组合都不会被误判为控制信息

怎么记不混

「组帧」要解决的核心矛盾：物理层送来的是连续比特流，接收方得知道一帧从哪开始、到哪结束。四种方法就是四种「划边界」的思路。

- **字符计数法**最简单，但它有个致命软肋：计数字段一旦传错，后面所有帧的边界全跟着错位，而且**无法重新同步**——其余三种都能靠重新找定界标志恢复。
- **零比特填充法**是 408 最爱考的：Flag 是 `01111110`，含 6 个连续 1。为防止数据里冒出 6 个连续 1 撞标志，发送方在**连续 5 个 1 后就插 0**（不是 6 个 1 后才插，那就晚了）；接收方见 5 个 1 就删掉其后的 0。
- **字符填充法**记一个坑：ESC 本身出现在数据里时也必须转义成 `ESC ESC`，否则接收方分不清「转义后的 EOT」还是「数据中的 ESC + 真结束标志」。

减法红线

具体「给一段比特串写出填充结果」的逐位演算交给练习题，卡片只锚「哪种用填充、填什么字符 / 几个 1 后插 0」这一句结论。

> 展开博客精讲：封装成帧与透明传输（ `/cn-blog/posts/datalink/framing.html` ）

校验码 冷门 对比表

码距 d 与检错纠错能力（结论公式，不背具体演算）：

- 检 e 位错：**d ≥ e + 1**
- 纠 t 位错：**d ≥ 2t + 1**

海明码位数关系：设数据 n 位、校验 k 位，须满足

- $2^k \geq n + k + 1$（+1 留给「无错」状态；k 个校验位本身也可能出错）

三种校验码能力对照：

校验码	码距	能力
奇偶校验	d = 2	**只能检奇数个错**，不能纠错，检不出偶数个错
海明码	d = 3	**纠 1 位 或 检 2 位**（二选一）；加全校验位 P_0 升 d=4 即 SEC-DED（同时检 2 纠 1）
CRC	视生成多项式	**检错能力强**（含突发错），本身不纠错，配重传实现纠错

校验码的考点拆成两半：**码距/位数关系这些结论**该背（套公式即得，选择题送分），而**海明码校验位排第几位、某数据串的海明码、CRC 模 2 除法、奇偶位具体取值**全是演算，交给题和可视化，这张卡一律不碰。

怎么记不混三个公式：

- 检错门槛低、纠错门槛高：检 e 位只要 `d ≥ e+1`，纠 t 位要 `d ≥ 2t+1`。直观理解——要「纠」必须能唯一定位错在哪，所以码距要求翻倍还多 1。
- 海明码 `2^k ≥ n+k+1`：右边为什么是 `n+k+1` 而不是 `n`——要区分的状态是「n 个数据位出错 + k 个校验位出错 + 1 个无错」，故障字必须能编码这么多种情况。这是 co-2013-15 的核心陷阱（只记 `2^k≥n` 会少算）。

三码能力一句话区分：

- **奇偶校验**最弱：码距 2，只能检出奇数个错（偶数个错奇偶性不变就漏了），永远不能纠错。
- **海明码**码距 3，「纠 1」和「检 2」是二选一不能同时；想同时检 2 纠 1 要再加一位全校验位 P_0（SEC-DED，码距升到 4）。
- **CRC** 检错能力最强、尤其能检突发错，但它本身只检不纠，工程上配「检出即重传」来兜纠错。

海明码的校验位位置、分组、故障字定位，以及 CRC 的模 2 除法过程，都是计算流程，看下方可视化或博客逐步推演；卡里只锚「码距公式 + 位数关系 + 三码能力」这些结论。

信道划分 MAC 冷门 对比表

- 四种信道划分（共享一条链路、各占一个「维度」）：

方式	划分维度	是否同时发	共享频带	典型应用
FDM 频分	频率（各占一段子频带）	同时	否	广播、ADSL
TDM 时分	时间（各占一个时隙）	轮流	是（轮流用全部）	T1/E1 线路
WDM 波分	波长（= 光纤里的 FDM）	同时	否	光纤骨干网
CDM/CDMA 码分	编码（正交码片）	同时	是（全频全时共享）	CDMA 移动通信

- 一句话锚点：谁按频分 / 时分 / 波分 / 码分

怎么记不混

- **WDM 本质就是光纤里的 FDM**——波长和频率一一对应（c = λf）。选择题里看到「光纤」选 WDM、看到「电缆/无线」选 FDM。
- **TDM 的「同步」**指收发端时钟同步，接收端才知道当前时隙归谁；改进版 STDM（统计/异步时分）按需分配时隙、利用率更高，代价是每时隙要带地址信息。
- **CDM 是真·同时同频**：所有用户用同一频带、同一时间一起发，靠码片序列的**正交性**在接收端分离。这是和 FDM/TDM 最本质的区别（编码维度，不切频也不切时）。

减法红线

CDMA 码片序列的内积/正交计算是算题，**不背**（交给练习题 + CDMA 计算器可视化）。卡片只锚「谁按频/时/波/码分 + 各自适用介质场景」。

> 展开博客精讲：信道复用技术 FDM/TDM/WDM/CDM（ `/cn-blog/posts/physical/multiplexing.html` ）

轮询访问 / 令牌 冷门 定义锚点

- 令牌传递 = **确定性**介质访问控制：网络里有一个特殊控制帧「令牌」，**只有持令牌的站点才能发数据**，发完再把令牌交给下一站。
- 本质特征：**无冲突**（靠令牌轮流授权，不靠争用）、**有最大等待上界**（令牌转一圈的时间，时延确定，适合实时场景）。
- 与随机访问（CSMA/CD）的根本区别：**确定性 / 有序 vs 竞争性 / 争用**。
- 负载适应性：**轻负载效率低**（令牌空转浪费时间）、**重负载效率高**（无冲突，利用率可接近 100%）——与 CSMA/CD 正好相反。

怎么记不混

令牌传递是低频考点，记住一句定义 + 一组对比即可，不涉及计算。

- **核心一句话**：谁拿到令牌谁才能发，发完传给下一站，所以永远不会两站同时发 → **没有冲突**。
- **「确定性 vs 竞争性」是与 CSMA/CD 对比的题眼**：令牌环是排好队轮流来（确定性），CSMA/CD 是抢着发、撞了再退（竞争性）。由此推出两条结论——令牌环有「最大等待时间上界」（适合实时），CSMA/CD 的时延不确定（冲突导致）。
- **轻 / 重负载性能对比常考**：轻负载时令牌大部分时间在空转，反而不如 CSMA/CD「信道空闲直接发」来得快；重负载时 CSMA/CD 冲突频繁效率骤降，而令牌环依然无冲突、利用率高。记成「**令牌环越忙越香，CSMA/CD 越忙越糟**」。

  数据帧绕环一圈回到发送方由发送方移除、令牌丢失需监控站恢复等细节属机制理解，要展开看「展开博客精讲」（ `/cn-blog/posts/datalink/token-passing.html` ）。

广域网与 PPP 冷门 定义锚点

- PPP（点对点协议）= 使用最广的**点对点**数据链路层协议（拨号 / 宽带接入 ISP 用它）。
- 四大特征（死记）：**面向字节**、**只支持点对点**（不支持多点/广播）、**不提供可靠传输**（无序号、无确认，错帧直接丢，可靠性交上层）、**不提供流量控制**。
- **三个组成部分**：① PPP 帧格式 ② **LCP**（链路控制协议，建立/配置/测试/释放链路）③ **NCP**（网络控制协议，为各网络层协议配置连接，如 IP 对应 IPCP）。
- 帧字段固定值：标志 F = **0x7E**、地址 A = **0xFF**（点对点不需寻址）、控制 C = **0x03**（无编号帧 → 无序号无确认）。
- 透明传输：**异步用字节填充（0x7D 转义）、同步用零比特填充（5 个 1 后插 0）**。

怎么记不混

- **PPP 不可靠是核心考点**：控制字段固定 0x03（无编号帧），没有序号和确认机制，帧出错直接丢弃，由上层（如 TCP）兜可靠性。**别因为它有 FCS 就以为可靠**——FCS 只能检错、不能纠错也不会重传。
- **LCP vs NCP 别搞混**：LCP 管**链路层**的事（建/拆链路、协商 MRU、认证方式）；NCP 管**网络层**的事（如 IPCP 分配 IP 地址、配 DNS）。两者工作在不同阶段：先 LCP 建链路、（可选认证）、再 NCP 配网络层。
- **地址/控制字段是「摆设」**：A 固定 0xFF、C 固定 0x03，因为点对点不需要地址区分多终端、也不用序号确认。但它们仍是帧格式的一部分，不能随意省略。
- **两种透明传输对应两种传输模式**：异步（面向字节）→ 0x7D 字节填充；同步（面向比特）→ 与 HDLC 一样的零比特填充。

  协议字段各取值（0x0021 IP / 0xC021 LCP…）、PAP/CHAP 认证、PPP 状态机各阶段属细节，看「展开博客精讲」（ `/cn-blog/posts/datalink/ppp.html` ）；字节填充/零比特填充的具体演算交给练习。

网络层 11 张

子网划分 / 子网掩码 / CIDR / NAT 高频 硬结论数字

- CIDR 记法 a.b.c.d/x：前 x 位为网络前缀，地址块共 2^(32-x) 个地址
- 可用主机数 = 2^主机位数 − 2（减去主机号全 0=网络地址、全 1=广播地址）——结论，背
- /x ↔ 点分掩码常见值（背熟，做题直接写）：
  /24=…0 · /25=…128 · /26=…192 · /27=…224 · /28=…240 · /29=…248 · /30=…252
- 网络地址 = IP 按位与子网掩码；最长前缀匹配 = 多条路由命中时取前缀最长（最具体）那条
- NAT：私网→公网地址转换，装在内外网边界路由器；NAPT 同时换 IP+端口，实现多台内网主机共享一个公网 IP
- 私有地址段：10.0.0.0/8 · 172.16~31.0.0/12 · 192.168.0.0/16

子网划分是 408 网络层第一金矿，但**借位、网络地址/广播地址求解、逐位与运算这些计算过程交给题**——本卡只锚「套公式即得」的结论。

可用主机数 = $2^n - 2$ 是必须脱手的结论（n=主机位数）。减 2 是因为主机号全 0 是网络地址、全 1 是直接广播地址，都不能分给主机。注意这是**结论不是计算**：知道掩码 → 数出主机位 → 直接代公式。

/x ↔ 点分掩码互换也是结论级速记，做题直接写不用每次算：**/26=255.255.255.192、/27=…224、/28=…240** 这一串背熟。规律是从 /24 起每多 1 位，最后一字节就把下一个高位置 1（128→192→224→240→248→252）。

最长前缀匹配：一个目的 IP 可能命中路由表多条记录，取**掩码最长（最具体）**的那条。「最长」指掩码位数，不是匹配字节数——/27 比 /24「长」。默认路由 0.0.0.0/0 是最不具体的兜底。

CIDR 记法 a.b.c.d/x：前 x 位网络前缀，地址块含 2^(32−x) 个地址（如 /22 = 1024 个 = 4 个 C 类）。

NAT 装在内外网边界路由器，把私网地址换成公网地址；我们平时说的 NAT 多指 **NAPT**——同时换 IP 和端口，让多台内网主机靠不同端口共享一个公网 IP。私有地址三段记死：10 / 172.16~31 / 192.168。

> 给 IP+掩码求网络地址/广播地址/主机范围、VLSM 变长划分、路由聚合这些**演算属题不属卡**；想动手算，点开子网计算器。

ARP / DHCP / ICMP 中频 对比表

- 层归属（陷阱）：ARP、ICMP 均属【网络层】；DHCP 属应用层（用 UDP）
- ARP：IP→MAC 映射；同网段「请求广播、应答单播」；有缓存；不能跨路由器（隔广播域）
- DHCP：动态分配 IP，四步 DORA = Discover→Offer→Request→ACK；走 UDP，服务器 67 / 客户端 68；Discover/Request 一定广播
- ICMP：差错报告 + 询问两类报文；ping=回送请求/回答(类型 8/0)；traceroute=TTL 递增 + 超时报文
- ICMP「四不发」差错报文：① 对 ICMP 差错报文本身 ② 对非第一个分片 ③ 对组播地址 ④ 对特殊地址(127.x / 0.0.0.0)

网络层三个辅助协议的对比。两条最容易踩的陷阱：**层归属**和 **ICMP 四不发**。

层归属钉死：ARP 和 ICMP 都属网络层。 ARP 报文虽直接封装在以太网帧里（类型 0x0806）、ICMP 报文虽封装在 IP 数据报数据部分（协议字段=1），但二者都是**网络层**协议——这是高频陷阱，「ICMP 是传输层」「ping 用 UDP」都是错的，ping 直接用 ICMP 不经传输层。而 **DHCP 是应用层**协议（用 UDP 67/68 承载）。

ARP：解决 IP→MAC。同网段「**请求广播、应答单播**」（请求时不知目标 MAC 只能广播；应答时请求报文里已带请求方 MAC，故单播）。跨子网时解析的是**默认网关的 MAC**，不是目的主机的；全程 IP 端到端不变、MAC 逐跳改变。ARP 不能跨路由器（路由器隔离广播域）。

DHCP：四步 **DORA**（Discover→Offer→Request→ACK）。Discover 和 Request 一定是广播（客户端此时还没 IP，且 Request 要通知未被选中的服务器回收地址）；用 UDP 因为没 IP 建不了 TCP 连接、且要广播。

ICMP：分差错报告和询问两类。ping 用回送请求/回答（类型 8/0）；traceroute 靠 TTL 递增触发中间路由器回超时报文。**「四不发」差错报文**记死：① 对 ICMP 差错报文本身不再发（防无限循环）② 对非第一个分片不发 ③ 对组播地址不发 ④ 对特殊地址（127.x.x.x、0.0.0.0）不发。

DHCP 报文里源/目的 IP 的具体填法、ARP 跨子网逐跳 MAC 替换的动画，点「展开博客精讲」或 ARP 可视化。

IPv4 分组与地址 中频 硬结论数字

- 首部固定 20B（可选部分 0~40B，故首部总 20~60B）；首部校验和只校验首部、不校验数据
- 协议字段（指上层协议）：TCP=6 · UDP=17 · ICMP=1 · OSPF=89
- TTL：每过一个路由器减 1，减到 0 丢弃 → 防止分组在网络中无限循环（防环）；最大 255
- IP 分类：A/B/C 类首位 0 / 10 / 110，网络号位数 8 / 16 / 24；A 类首字节 1~126、B 类 128~191、C 类 192~223
- 特殊地址：0.0.0.0=本网本机 · 127.x.x.x=环回 · 255.255.255.255=受限广播(路由器不转发) · 主机号全 0=网络地址 · 主机号全 1=直接广播
- 私有地址：10.0.0.0/8 · 172.16~31.0.0/12 · 192.168.0.0/16

IPv4 分组与地址的硬结论速记。**字段位数/分片偏移/校验和的具体演算交给题**，这里只锚结论。

首部固定 20 字节——这是分片、求数据部分长度等一切计算的起点常量。首部校验和只覆盖首部、不覆盖数据（IP 把数据校验交给上层 TCP/UDP）。

协议字段记死四个数：TCP=6、UDP=17、ICMP=1、OSPF=89。选择题里偶尔直接给字段值让你判上层协议。

TTL 的本质是防环：每经过一个路由器减 1，减到 0 就丢弃并（可）回 ICMP 超时报文。它防的是路由出错时分组在网络里无限循环，**不是限制总跳数上限**那种业务语义。

地址分类靠首字节速判：1~126 是 A 类、128~191 是 B 类、192~223 是 C 类。注意 **127 开头不是 A 类**（是环回），0 开头保留。

特殊地址别背混：主机号全 0 是网络地址（标识网络本身）、主机号全 1 是直接广播地址，这两个都不能分给主机；255.255.255.255 是受限广播（只在本网广播，路由器不转发）；0.0.0.0 是启动时的「本网本主机」。

> 分片偏移单位 8B、标志位 DF/MF、片数计算这类**演算属题不属卡**；想看分片切分的交互动画，点「展开博客精讲」。

网络层功能 中频 对比表

- 三大功能：路由（选路，控制面）+ 转发（查表送出，数据面）+ 异构网络互联
- 路由 ≠ 转发：路由=算路径并更新路由表（全网拓扑、慢、秒级收敛）；转发=对单个分组查表选下一跳（单台路由器内、极快、每包微秒级）
- 异构网络互联：IP 当统一「普通话」，靠 IP 分片 / ARP / 路由器把不同链路层拼成一张网
- SDN：把控制面（算路）从设备里抽出来集中到控制器，数据面只按流表「匹配+动作」——考点是「控制面与数据面分离」
- 两种服务模型：数据报（无连接、每包独立寻址、可能失序、IP 网络）vs 虚电路（先建连、走同一路、不失序、VCID 局部标识）

网络层「做什么」的通论。核心是把**路由（routing）**和**转发（forwarding）**两个词分清——这是 408 一旦混用就扣分的辨析点。

路由是控制面、转发是数据面。 路由由路由协议（RIP/OSPF/BGP）协作，综合全网拓扑算出每个目的网络走哪条路，写进路由表，时间尺度慢；转发是对已到达的每个分组查表选下一跳、改 TTL、重新封装转出，时间尺度极快。一句话：**路由算路、转发执行**。

IP 不可靠不代表不会丢。 「不可靠」是「丢了不重传、不告知」，拥塞时路由器主动丢 IP 分组是网络层正常行为。路由器属于网络层，不做端到端可靠性——这正是路由器「会做什么、不会做什么」类选择题的考核点。

异构网络互联靠 IP 这个统一网络层协议向上屏蔽底层差异（沙漏模型的腰部），手段是 IP 分片（适配不同 MTU）、ARP（IP→链路层地址）、路由器（不同链路类型间转换+转发）。

数据报 vs 虚电路只需会辨析：IP 走数据报模型（每包独立查表、可能失序）；虚电路要先建连、同一连接走同一路、不失序，VCID 是每段链路局部的标识（和「IP 地址端到端不变」对照）。408 实务基本只考数据报，虚电路用来做对比。

> 想看路由/转发分工与数据报/虚电路对照的交互图，点「展开博客精讲」。

路由协议（RIP/OSPF/BGP） 中频 对比表

三大路由协议对比（域内 IGP / 域间 EGP）：

协议	类别	算法	承载（死记）	度量
RIP	IGP（域内）	距离向量 DV	**UDP 端口 520**	跳数，最大 **15**（16=不可达）
OSPF	IGP（域内）	链路状态 LS + Dijkstra	**协议号 89，直接封装在 IP 数据报里，无端口**	链路代价
BGP	EGP（域间，AS 之间）	路径向量	**TCP 端口 179**	策略（非最短路径）

- 承载三件套（power user 试金石）：**RIP=UDP 520 / OSPF=协议号 89 直封 IP·无端口 / BGP=TCP 179**。OSPF 既不用 TCP 也不用 UDP，是唯一没有端口号的。
- RIP 最大跳数 **15**，跳数 16 即"不可达"（防无穷计数）。
- IGP（RIP/OSPF）管 AS 内部；EGP（BGP）管 AS 之间。

三协议的"承载在什么之上"是选择题最爱的陷阱，尤其 **OSPF 易被误记成有端口号**——它的报文直接封装在 IP 数据报里，协议号 89，既不走 TCP 也不走 UDP，没有任何端口。把这三句钉死：RIP 走 UDP 520、OSPF 协议号 89 无端口、BGP 走 TCP 179。

怎么记不混：

- 按"可靠性需求"反推承载：RIP 报文短、定期广播、丢了下轮再来，所以用最轻的 **UDP**；BGP 在 AS 之间传海量路由、对可靠性要求极高，所以用 **TCP**；OSPF 自己用洪泛 + LSAck 做可靠传输，不需要传输层兜底，干脆**直接骑在 IP 上**。
- 度量各不同：RIP 数**跳数**（最大 15），OSPF 算**链路代价**（可基于带宽），BGP 看**策略**（不追求最短路径，可能为绕开某 AS 故意选长路径）。
- 域内 vs 域间：RIP、OSPF 是 IGP（一个 AS 内部用）；BGP 是 EGP（AS 与 AS 之间用）。

  RIP 的路由表"距离 +1、下一跳相同则无条件更新"、OSPF 的 Dijkstra 算路属于计算/推导，交给题和可视化。卡里只锚结论，要看更新过程点「展开博客精讲」或下方 RIP 可视化。

路由器组成与功能 低频 定义锚点

- 两大功能：**分组转发**（查转发表 + 最长前缀匹配，数据平面）+ **路由计算**（运行路由协议维护路由表，控制平面）。
- 组成：**输入端口 + 交换结构 + 输出端口**（数据平面）+ **路由选择处理机**（控制平面）。
- 工作在网络层，**既隔离冲突域、也隔离广播域**（对比交换机只隔冲突域，不隔广播域）——因为路由器**不转发广播帧**。
- 路由表（控制平面，算法算出）≠ 转发表（数据平面，由路由表导出供高速查表）。

路由器组成偶尔出选择题，冲突域/广播域分割能力是高频送分点（2010、2020 真题都直接考过设备的隔离能力）。

钉死两条锚点：

- **隔离能力**：集线器/中继器啥都不隔；网桥/交换机只隔**冲突域**；路由器**冲突域、广播域都隔**。路由器能隔广播域的根本原因是它工作在网络层、**不转发广播帧**。
- **转发 vs 路由**：转发是单台路由器查转发表把分组从合适端口送出（数据平面）；路由是多台路由器协同跑路由协议生成路由表（控制平面）。路由表和转发表不是同一张——前者"怎么算出来"，后者"实际查哪张"。

  最长前缀匹配的逐位比对、默认路由 0.0.0.0/0 的命中判断属于计算/操作细节，交给题；要看转发流程点「展开博客精讲」或下方路由器结构可视化。

路由算法 低频 对比表

- 三个正交分类轴：静态/动态 · 距离向量(DV)/链路状态(LS) · 平面/层次
- 距离向量(DV)：算法=Bellman-Ford；只知「邻居说它到哪、多远」；发整张距离向量、只给直连邻居；周期性(+触发)更新；收敛慢、有「计数到无穷」；典型 RIP
- 链路状态(LS)：算法=Dijkstra；每台拥有全网拓扑数据库；发自己的链路状态、洪泛到全 AS；链路变化即发；收敛快；典型 OSPF
- DV 递推：D_x(y)=min over 邻居v [c(x,v)+D_v(y)]（公式名+定性，数值演算交给题）
- 防环（DV）：水平分割 / 毒性逆转 / 触发更新
- 层次路由：AS 内用 IGP（RIP/OSPF）、AS 间用 EGP（BGP，路径向量带 AS 序列防环）

路由算法的「算法层面」通论，和具体协议（RIP/OSPF/BGP）相辅相成——**算法是骨架，协议是肉**。

DV 与 LS 的对比是核心。 一句话记忆：**「DV 是盲人摸象，LS 是上帝视角」**。

- **距离向量 DV**：每台路由器只知道「经过某邻居能到哪、多少代价」，不知道邻居之间的拓扑；周期性把**整张距离向量**发给**直连邻居**。简单，但收敛慢、会「计数到无穷」（链路断了，距离被一跳跳累加放大，直到撞上不可达上限才停）。
- **链路状态 LS**：每台只把**自己的本地链路状态**洪泛（flooding）到全 AS，于是每台都拥有**相同的全网拓扑数据库**，各自跑 Dijkstra 算最短路。收敛快。关键不变量：**所有路由器的链路状态数据库必须一致**，否则会算出不一致路径而成环。

三个分类轴是正交的：同一协议可被多个轴标记，比如「RIP = 动态 + 距离向量 + 域内」。所以「AS 用什么协议」不是单选题，实务里一个 AS 内可能 OSPF + 静态 + 对外 BGP 并存。

层次路由：因特网太大，全网跑一个协议会内存爆炸。按自治系统 AS 分层——AS 内用 **IGP**（RIP/OSPF），AS 间用 **EGP**（BGP）。BGP 用**路径向量**，每条路由带完整 AS 序列，看到自己 AS 号在序列里就丢弃以防环。

  DV 递推、Dijkstra 最短路这类**计算交给题**；想看 DV/LS 逐步演化的交互图，点「展开博客精讲」。

拥塞控制（网络层视角） 冷门 易错辨析

- 拥塞 = 网络整体负载超过容量 → 丢包增多、时延增大、吞吐量反而下降
- 拥塞控制 vs 流量控制（高频辨析）：
  - 拥塞控制 = 全局性，防整个网络过载，涉及所有主机+路由器
  - 流量控制 = 端到端，防发送方淹没接收方，只涉及收发双方
- 控制方式：开环（预先设计好、运行中不调整）vs 闭环（运行中监测拥塞、反馈调节）
- 拥塞是「全局」问题：单看某条链路负载不够，要看全网

网络层视角的拥塞控制，**只背概念辨析，不背 TCP 拥塞窗口的数值演算**（那归 `tcp-congestion` ）。

拥塞控制 ≠ 流量控制是本卡的命门。 两者都「让发送方慢下来」，但目的和范围不同：

- **拥塞控制**针对**整个网络**负载过大，是**全局**行为，涉及所有主机和路由器；它解决的是「网络这条大马路堵车了」。
- **流量控制**针对**单条端到端连接**里发送方发太快、接收方来不及收，只涉及收发两端；它解决的是「我说话太快你记不下来」。

判别口诀：**问题出在「网络」就是拥塞控制，问题出在「接收方」就是流量控制。**

拥塞是**全局性**问题——网络中链路容量、路由器缓存和处理能力都有限，总负载超过承载力时性能急剧下降。所以只盯一条链路看不出拥塞，要看全网。

开环 vs 闭环：开环是事先把网络设计好、运行中不再临时调整（静态预防）；闭环是运行中实时监测拥塞迹象、把信号反馈回去动态调节（如路由器丢包/置标志通知源主机降速）。

> TCP 怎么用慢开始/拥塞避免具体调窗口，是传输层的事（见 `tcp-congestion` 卡），本卡不碰窗口演算。

IP 组播 冷门 定义锚点

- 组播 = 一对多（只发一份，沿途分岔点才复制）；用 **D 类地址**标识组，范围 **224.0.0.0 ~ 239.255.255.255**（前 4 位 1110）。
- 组播地址**只能做目的地址**，不能做源地址（源仍是普通单播 IP）。
- 只用于 **UDP**，不用于 TCP（TCP 是一对一连接）。
- **IGMP**（网际组管理协议，属网络层）作用：让本地组播路由器知道**本子网内**有哪些主机加入了哪些组；只在本地子网工作，不管全网。

组播在选择题里偶尔露面，记住"D 类地址范围 + IGMP 干什么"这条锚点即可。

几个易混点：

- **D 类地址 224~239**——前 4 位固定 1110，这段地址专用于标识组播组。
- **组播地址只能当目的地址**——一个分组的源永远是发送方的普通单播 IP，目的才可能是 D 类组播地址。
- **只走 UDP**——组播是一对多，而 TCP 建的是一对一端到端连接，天然不支持组播。
- **IGMP 只在本地子网**——它只负责让本地组播路由器掌握本子网的成员情况，不知道全网组成员总数和分布。

> 组播 MAC 映射（28 位映射到低 23 位、32 个 IP 映射到同一 MAC 的多对一计算）是算题，不背中间过程；要看映射规则点「展开博客精讲」。

IPv6 冷门 硬结论数字

- 地址 **128 位**，**冒分十六进制**表示（8 组 ×16 位）；前导零可省，连续全零组用 **::** 替代（**全地址只能用一次**）。
- 首部**固定 40 字节**（IPv4 是 20~60 可变）。
- 三大简化：**取消首部校验和**、取消首部长度字段、取消分片相关字段（改用扩展首部链）。
- 三种地址：单播 / 组播 / 任播；**没有广播**（广播功能由组播替代）。
- 路由器**不分片**，只在源主机分片（包太大则回 ICMPv6"分组太大"）。
- 过渡技术三种：**双栈 / 隧道 / 翻译**。

IPv6 是中低频考点，选择题主要考它与 IPv4 的差异，记住几个"硬数字 + 反直觉点"就够，不必深挖细节。

最容易被挖坑的几处：

- **取消了首部校验和**——IPv4 每跳都要重算校验和太费时，IPv6 认为链路层有 CRC、上层有 TCP/UDP 校验和，网络层这层是多余的，直接砍掉。
- **没有广播**——IPv6 只有单播/组播/任播，原来 IPv4 广播的活由组播接管。这是高频陷阱。
- **:: 只能出现一次**——出现两次就无法确定各自省略了多少组零，是非法缩写。
- **首部固定 40 字节**——正因为定长，连"首部长度"字段都不需要了。

  IPv6 地址缩写的具体展开、扩展首部链的组装属于"会写就行"的操作，不必背中间过程；要看地址格式演示点「展开博客精讲」。

移动 IP 冷门 定义锚点

- 目标：移动主机**切换网络时归属 IP 不变**，TCP 连接不中断，对上层透明。
- 三实体：**移动节点**、**归属代理 HA**（归属网络路由器，截获并隧道转发）、**外地代理 FA**（被访网络路由器，分配转交地址并解封装交付）。
- 两地址：**归属地址**（永久不变）、**转交地址 CoA**（随被访网络动态变化）。

移动 IP 是 408 的极低频考点，记住"为什么需要它 + 三实体 + 两地址"这一句就够，别堆细节。

一句话理解：正常换网络就得换 IP，TCP 连接随之断开；移动 IP 让你换到哪儿都保持原来那个**归属地址**，靠**归属代理**截获发给你的包、隧道转发到你当前的**转交地址**，连接就不会断。

> 通信者→归属代理→隧道→外地代理→移动节点的转发路径、以及"发送不经归属代理"的不对称性属于流程细节，要复述全过程点「展开博客精讲」。

传输层 6 张

TCP 连接管理 高频 步骤口诀

三次握手（建连）：

1. SYN（A→B，SYN=1, seq=x）
2. SYN-ACK（B→A，SYN=1, ACK=1, seq=y, ack=x+1）
3. ACK（A→B，ACK=1, seq=x+1, ack=y+1）→ ESTABLISHED

四次挥手（释放）：

1. FIN（A→B）→ A: FIN_WAIT_1
2. ACK（B→A）→ B: CLOSE_WAIT（半关闭，B 还能发）、A: FIN_WAIT_2
3. FIN（B→A，B 数据发完）→ B: LAST_ACK
4. ACK（A→B）→ A: TIME_WAIT，等 **2MSL（≈4 min）** → CLOSED

硬结论：

- TCP 固定首部 **20 B**；6 个标志位：**SYN / ACK / FIN / RST / PSH / URG**。
- SYN、FIN 各消耗 1 个序号；纯 ACK 不消耗序号。

TCP 连接管理是 cn-transport 第一高频考点（命题 16 次居首），三次握手/四次挥手 + 状态机几乎每年都考，必须脱手默写。这张是步骤口诀卡——core 背骨架，下面记**每一步为什么**。

三次握手为什么是三次、不是两次：核心是**防止已失效的连接请求突然又到达服务器**，导致服务器误建连接、白白占资源。换个本质角度：三次握手让双方都确认对方的**收发能力都正常**——两次只能让 B 确认 A 能发，没法让 B 确认 A 能收。
（顺带：第一次握手 SYN=1 但 **ACK=0**，因为还没收到对方数据，确认号无意义；前两次不能带数据，**第三次可以带数据**。）

四次挥手为什么是四次、不是三次：TCP 是全双工，两个方向各自关闭。B 收到 A 的 FIN 后**可能还有数据没发完**，所以先回一个 ACK（"知道你要关了"），等自己数据发完再发 FIN——**B 的 ACK 与 FIN 不能合并**，这就比握手多了一步。第二步之后到第三步之间叫**半关闭**：A→B 方向已关，B→A 方向还能发数据。

为什么 TIME_WAIT 要等 2MSL（≈4 min，MSL 典型 2 min）：两个原因——① **保证最后一个 ACK 能到 B**：万一 ACK 丢了，B 会重传 FIN，A 还在 TIME_WAIT 就能补发 ACK；② **让本次连接的残留报文段全部超时消失**，避免污染下一条新连接。这是经典口述题，命题组每隔几年用一次。

易混锚点：CLOSE_WAIT 是**被动关闭方**的状态（看到大量 CLOSE_WAIT 通常是程序没调 close）；四次挥手第三步 B 的 seq 不一定等于 v+1（半关闭期间可能又发了数据）；RST=1 表示严重错误立即拆连接，收到方直接关、不回 ACK。

> 序号/确认号的逐步推算（SYN/FIN 各 +1、ack = 对方 seq + 数据长度）是计算题，交给练与可视化，卡里只锚"SYN/FIN 各消耗 1 序号"这条规则。点「展开博客精讲」看带 seq/ack 的完整推导表与状态转换图。

TCP 拥塞控制 中频 步骤口诀

四阶段口诀（cwnd 单位 MSS）：

1. **慢开始**：cwnd 从 1 起，每 RTT **翻倍**（指数增）→ 到 ssthresh 转②
2. **拥塞避免**：每 RTT **+1**（线性增，加法增大）
3. **快重传**：收 3 个重复 ACK，立即重传不等超时
4. **快恢复**：紧接快重传，**ssthresh = cwnd/2，cwnd = ssthresh**，进拥塞避免

AIMD = 加法增大（+1）+ 乘法减小（/2）。

★ 两种"减速"必须分清（高频失分点）：

触发	ssthresh	cwnd	下一步
超时	cwnd/2	**= 1**	回慢开始
3 个重复 ACK	cwnd/2	**= ssthresh（不回 1）**	拥塞避免

> 标准 RFC Reno 快恢复会把 cwnd 充气为 ssthresh + 3MSS（计入触发的 3 个失序段）；408 一律按 **cwnd = ssthresh** 答题，做题就用上表，+3MSS 只作了解。

TCP 拥塞控制是传输层绝对核心，几乎每年选择题或大题出现。这是步骤口诀卡——core 背骨架，下面记**每一步为什么、以及最容易记反的地方**。拥塞是**全局性**问题（网络整体负载超容量导致丢包/时延飙升），区别于只管收发双方的流量控制。

慢开始为什么叫"慢"：不是增长慢——它是**指数增长**（1→2→4→8...），增速很快。"慢"是指**起点低**：从 1 个 MSS 起步试探网络容量，相对于一上来就开满窗口而言起步慢。到 ssthresh（慢开始门限）就转拥塞避免。

拥塞避免为什么改线性：cwnd 大了再翻倍风险太高，改成每 RTT **+1**（加法增大），更谨慎地逼近拥塞点。它不是"完全避免拥塞"，只是降低触发概率。

快重传 → 快恢复的逻辑：能收到 3 个重复 ACK，说明网络只是**轻度拥塞还没堵死**（否则连重复 ACK 都收不到），没必要像超时那样把 cwnd 砸回 1 重新慢开始——所以快恢复只把 cwnd 减半后直接进拥塞避免，恢复快。这是 **Reno 相对 Tahoe 的关键改进**（Tahoe 把 3 个重复 ACK 也当超时处理、回 cwnd=1），**408 默认考 Reno**。

最容易丢分的辨析（core 里的表，再强调一遍）：

- **超时**（明确的严重拥塞信号）→ ssthresh = cwnd/2，**cwnd = 1**，回慢开始。
- **3 个重复 ACK**（轻度拥塞）→ ssthresh = cwnd/2，**cwnd = ssthresh，不回 1**，进拥塞避免。

记反这两个"减速幅度"是命题组最爱挖的坑。另注意**顺序**：先更新 ssthresh，再设 cwnd，别先改 cwnd 再算 ssthresh。

两个边角锚点：cwnd = ssthresh 时执行哪个算法，不同教材说法不一（常见处理是执行拥塞避免），做题看是否有特别说明；实际发送窗口 = min(cwnd, rwnd)，题目只提拥塞控制时默认 rwnd 足够大、发送窗口 = cwnd。

> 给定 ssthresh 和事件序列逐轮算 cwnd 是计算题，交给练与可视化，卡里只锚窗口变化的"定性规律 + 触发动作"。点「展开博客精讲」看完整 cwnd 变化曲线与逐轮表。

TCP 可靠传输 中频 定义锚点

TCP 在不可靠的 IP 之上靠四机制实现可靠传输：

1. **字节编号 + 确认**：每个字节编号；ack = 期望收到的下一个字节号。
2. **超时重传**：发出后启重传计时器，超时未确认就重传。
3. **校验和**：检测比特差错。
4. **滑动窗口**：发送窗口 = min(cwnd, rwnd)，动态调整。

关键锚点：

- **累积确认**：ack=501 表示 1~500 全收到（容错好，但不精确）。
- **确认号 = 期望收到的下一个字节**，不是已收到的最后一个（差一陷阱）。
- **快重传**：收到 3 个重复 ACK 立即重传，不等超时。
- **SACK（选择确认）**：选项字段报告已收的不连续块，最多 4 块。

TCP 可靠传输是 408 核心考点，但卡里**只锚机制名与含义**——序号/确认号的具体推算、RTO 公式计算都是算题，交给练，背中间值有害。

四个机制怎么各司其职：IP 层会丢、会乱序、会出错、会重复；TCP 用**字节编号 + 确认**让接收方告诉发送方收到哪了，用**超时重传**补丢失，用**校验和**抓比特差错，用**滑动窗口**控速率（同时服务流量控制和拥塞控制）。

累积确认 + 差一陷阱（最常错）：确认号 ack 表示"**期望收到的下一个字节编号**"——收到字节 1~500，回的是 **ack=501 不是 500**。累积确认的好处是简单、容错（中间几个 ACK 丢了，只要最后一个到，发送方就知道前面全收到）；坏处是不精确：1~500 和 601~700 都收到、缺 501~600 时，只能回 ack=501，发送方不知道 601~700 已到——这正是 **SACK** 要解决的，它在选项里直接报告不连续块边界，让发送方只补真正丢的部分。

快重传 vs 超时重传：不靠快重传时只能等超时定时器（往往设得偏长，白等）。快重传利用**冗余 ACK**提前发现丢包：接收方一收到失序报文就立刻重发上一个按序确认；发送方**收满 3 个重复 ACK** 就立即重传，不等超时。为什么是 3 个而不是 1~2 个？1~2 个重复 ACK 可能只是乱序，3 个才是较可靠的丢包判据。

别记反的两个点：超时重传不代表数据一定丢了（可能只是 ACK 丢或 RTT 抖动）；TCP 滑动窗口大小**不固定**（随 rwnd/cwnd 动态调），这点和某些链路层固定窗口协议不同。

> 序号编号举例、RTO = RTTs + 4×RTTd、Karn 算法都是计算/推导细节，卡里不背，点「展开博客精讲」看完整推导。

TCP 流量控制 低频 定义锚点

流量控制 = 防发送方发太快淹没接收方（端到端的速率匹配）。

- 机制：接收方用 ACK 里的**接收窗口 rwnd** 通告可接收量，发送方据此调发送窗口。
- 发送窗口 = **min(cwnd, rwnd)**；只看流控时 = rwnd。
- **零窗口死锁**：rwnd=0 后，接收方腾空发的窗口更新若丢失 → 双方互等。
- **解法 = 持续计时器**：发送方收到零窗口后启动，到期发**零窗口探测报文段（1 字节）**，逼接收方回报当前窗口。
- 辨析：**流量控制（端到端、防接收方溢出）vs 拥塞控制（全局、防网络过载）**，控制变量分别是 rwnd 和 cwnd。

TCP 流量控制是中高频考点，常和拥塞控制结合出题。要解决的问题很直接：**发送方发得太快、接收方缓冲区来不及处理就会溢出丢数据**，所以让发送方速率不超过接收方处理能力。

核心机制（背这两句）：接收方按缓冲区空闲量算出 **rwnd**（= 缓冲区大小 − 已收但应用层还没读走的量），在**每个 ACK** 里通告给发送方；发送方发送窗口取 **min(cwnd, rwnd)**。注意 rwnd **不是单调递减**的——应用层从缓冲区读走数据后，rwnd 会回升。

零窗口死锁与持续计时器（必考点）：当接收方缓冲区满，rwnd=0，发送方必须停发。问题在于：接收方后来腾出空间、发了个 rwnd>0 的通告，但这个通告**丢了**，就死锁了——A 等 B 说能发，B 等 A 发数据。解法是发送方启动**持续计时器**，到期主动发一个仅 1 字节的**零窗口探测报文段**，逼接收方回报当前窗口；窗口仍 0 就重置计时器再等。记牢：**持续计时器是发送方启动的，不是接收方**。

最高频辨析——流量控制 vs 拥塞控制：

	流量控制	拥塞控制
防什么	接收方被淹没	网络整体过载
范围	端到端（点对点）	全局
控制变量	rwnd	cwnd

判断瓶颈：rwnd < cwnd → 瓶颈在接收方；cwnd < rwnd → 瓶颈在网络；实际发送窗口总取较小值。

> 窗口随收发动态变化的逐步数值推算交给练与可视化，卡里只锚机制。Nagle 算法、糊涂窗口综合症了解即可，点「展开博客精讲」深入。

传输层功能 低频 硬结论数字

传输层两件事：① 复用/分用（用端口号把报文交给对的进程）② 端到端通信。

常用端口号 + 承载协议（按号排序，死记）：

端口	协议	传输层
20 / 21	FTP（数据 / 控制）	TCP
22	SSH	TCP
23	Telnet	TCP
25	SMTP（发邮件）	TCP
53	DNS	**UDP 为主 / TCP**
67 / 68	DHCP（服务器 / 客户端）	**UDP**
80	HTTP	TCP
110	POP3（收邮件）	TCP
143	IMAP（收邮件）	TCP
161	SNMP	**UDP**
443	HTTPS	TCP
520	RIP	**UDP**

- 复用（发送方）：多个进程共用传输层往下发；分用（接收方）：按**目的端口号**交给对的进程。
- 端口号 16 位（0~65535）；0~1023 是熟知端口。

端口号是 CN 最经典的硬结论卡，几乎年年在选择题里直接送分或当干扰项，必须连"承载在 TCP 还是 UDP"一起记牢——只记号码不记承载层，遇到"下列基于 UDP 的应用是"这类题就会翻车。

怎么记不混：

- 走 **UDP** 的少数派挑出来单背：**DNS（53，区域传送才用 TCP）**、**DHCP（67/68）**、**SNMP（161）**、**RIP（520）**。这几个的共同点是"报文短、一问一答、或要广播"，不值得为它建 TCP 连接。其余基本都走 TCP。
- 邮件三件套连方向一起记：**SMTP 25 发**、**POP3 110 收**、**IMAP 143 收**。
- FTP 是唯一占两个端口的：**21 控制连接（整条会话保持）**、**20 数据连接**，别记反。
- DHCP 是唯一占两个端口的 UDP 应用：**67 服务器**、**68 客户端**。

复用 vs 分用一句话：发送方多个进程的数据共用传输层下交（复用），接收方按报文里的**目的端口号**把它送给对应进程（分用）。2018-39 就直接考"UDP 分用依据哪个字段"——答案是目的端口号，不是源端口、不是校验和。

> 端口号本身是纯记忆点，没有计算。需要看一个报文从应用层到链路层逐层封装/分用的全过程，点「展开博客精讲」看「数据包的奇幻之旅」。

UDP 协议 低频 定义锚点

UDP 四字真言：**无连接 · 不可靠 · 面向报文**，首部固定 **8 B**。

- 首部 8 B = 源端口 + 目的端口 + 长度 + 校验和，各 2 B（对照 **TCP 首部最少 20 B**）。
- 无连接：发数据前不握手，无开销无时延。
- 不可靠：尽最大努力交付，无确认/无重传/不保序；无流量控制、无拥塞控制。
- 面向报文：对应用层报文不拆不合，保留边界（对照 TCP 面向字节流）。
- 支持一对一/一对多/多对一/多对多（可广播多播；TCP 只能一对一）。
- 校验和算时加 **12 B 伪首部**（含协议号 17），伪首部不真正发送。

UDP 单独出大题概率低，但几乎每年作为 TCP 题的干扰项出现，四个特点 + 首部 8 B 是必背锚点。它的设计哲学就一句话：在 IP 之上只加了**复用/分用**和**差错检测**两件最基本的事，其它全不管。

怎么记不混：

- **8 B 对照 TCP 20 B**：UDP 首部永远 8 字节、没有例外（没有可变选项字段）；TCP 首部固定部分 20 字节、还能带选项。这一对数字是高频送分点。
- "**长度**"字段是**首部 + 数据**的总长，不是只有数据；最小值就是 8（数据为空时）。
- "**面向报文**"意味着一次 `sendto` 对应一个完整 UDP 数据报，接收端一次恰好收一个——这和 TCP 的字节流要靠应用层自己切边界形成鲜明对比。
- UDP 校验和**可选**（可填 0），TCP 校验和**强制**。

为什么"不可靠"反而常被选用：报文短、一问一答、或要广播/实时的场景，建 TCP 连接不划算。典型走 UDP 的：**DNS、DHCP、SNMP、实时音视频（RTP）**。注意"不可靠"只是 UDP 协议层的特性，应用层（如 TFTP、QUIC）完全可以自己在 UDP 上实现可靠性。

> 校验和、伪首部的逐字节求和是计算过程，交给题与可视化，卡里只锚"加 12 B 伪首部、协议号 17"这个结论。点「展开博客精讲」看完整首部与校验流程。

应用层 5 张

DNS 系统 中频 步骤口诀

- 功能：域名 ↔ IP 映射；分布式数据库
- 传输层：**UDP 53 为主；区域传送 / 响应超 512B 用 TCP 53**
- 域名层级（右→左级别递减）：根 → 顶级 TLD → 权威；本地 DNS 不在此层次内
- 查询标配：**前段「主机→本地 DNS」用递归；后段「本地 DNS→根/TLD/权威」用迭代**
  - 递归 = 被查方帮你查到底；迭代 = 被查方只告诉你「下一个该问谁」
- 缓存：本地 DNS 缓存映射，带 TTL，大幅减少对根/TLD 的查询
- 记录：A=域名→IPv4、AAAA=→IPv6、CNAME=别名、MX=邮件、NS=权威服务器

怎么记不混

- **递归/迭代别搞反**：递归是「被查方替你跑完全程」，迭代是「被查方甩给你下一个地址自己跑」。标配组合 = 前递归（主机省事）+ 后迭代（根/TLD 负担轻）。
- **DNS 不是只用 UDP**：默认 UDP 53 一问一答省掉握手开销；但**区域传送（主辅同步）和响应超 512B 时切 TCP 53**——这是 power user 高频挖的坑。

常见坑

- 本地 DNS 服务器**独立于** 根/TLD/权威 这条层次链之外，只是代理主机去查。
- CNAME 给的是「另一个域名」而非 IP，还要再查那个域名的 A 记录。

子项「递归共发几条报文 / 迭代最少最多几次」属计算，交给题（2010-40·2016-40·2020-40）。展开博客精讲见 source_blogs。

电子邮件 中频 对比表

- 组成：UA（用户代理）+ MTA（邮件服务器）+ MAA（POP3/IMAP）
- 路径：UA --SMTP--> 发件 MTA --SMTP--> 收件 MTA --POP3/IMAP--> 收件 UA

协议	端口	方向	角色
SMTP	**25**	推（Push）	发送 + 服务器间转发
POP3	**110**	拉（Pull）	下载（删/留两模式），本地操作不同步
IMAP	**143**	拉（Pull）	服务器保留，多设备实时同步

- SMTP 只能传 **7 位 ASCII**；传中文/图片/视频靠 **MIME** 编码（Base64 等）

怎么记不混

- **发=推=SMTP，收=拉=POP3/IMAP**，别搞反。
- **发送阶段全程 SMTP**：UA→发件 MTA、发件 MTA→收件 MTA 都用 SMTP；只有最后「收件人读邮件」才用 POP3/IMAP。
- 端口三连：SMTP 25 / POP3 110 / IMAP 143。

常见坑

- SMTP 本身**不能传二进制**，靠 MIME 把非 ASCII 内容编成 ASCII（MIME 是 SMTP 的扩展，不是独立协议）。
- 基于 Web 的邮箱：用户⇔服务器走 **HTTP/HTTPS**，服务器之间仍用 **SMTP** 转发。

  展开博客精讲见 source_blogs。

WWW 与 HTTP 中频 对比表

- WWW 三大件：URL（标识资源）+ HTTP（传输）+ HTML（描述）；**WWW ≠ Internet**
- HTTP：TCP **80**（HTTPS **443**）；**无状态**（靠 Cookie/Session 保持）；请求-响应模型
- 方法：GET（取，幂等）/ POST（提交，不幂等）/ PUT / DELETE / HEAD
- 状态码五类：1xx 信息 / 2xx 成功 / 3xx 重定向 / 4xx 客户错 / 5xx 服务器错

码	含义（三态辨析）
301	永久重定向，资源已迁新 URL
302	临时重定向，下次仍访问原地址
304	未修改，可用本地缓存（非错误）

- 常考：200 / 301 / 302 / 304 / 400 / 403 / 404 / 500
- 连接：HTTP/1.0 **非持久**（每对象一条 TCP）；HTTP/1.1 **持久**（Keep-Alive）+ 流水线
- **报文结构**：请求报文 = 请求行（方法+URL+版本）+ 首部行 + 空行 + 实体；响应报文 = 状态行（版本+状态码+短语）+ 首部行 + 空行 + 实体
- **常考首部**：Host（HTTP/1.1 必需，托多站靠它区分）· Connection（close 非持久 / keep-alive 持久）· Cookie（会话标识，无状态靠它认人）· If-Modified-Since（条件 GET，配 304）

怎么记不混

- **301/302/304 三态**：301 永久（搜索引擎更新索引）、302 临时（下次还问原地址）、304 未改走缓存——现最易漏 302，把 301/302 当成一个。
- **304 不是错误**：是「条件 GET」的优化结果， `If-Modified-Since` 命中未改即回 304（无对象体），客户端继续用缓存。
- HTTP 是**无状态**不是「无连接」：HTTP/1.1 有持久连接（有连接），但每个请求互相独立（无状态）。

报文结构（cn-2015-40 真题就给了一段请求报文）

请求报文 = 请求行 + 首部行 + 空行 + 实体主体。请求行 = 方法 + URL + 版本；GET 的实体主体通常为空，POST 把表单数据放实体主体。例（2015-40 原题）：

```
GET /index.html HTTP/1.1
Host: www.test.edu.cn
Connection: close
Cookie: 123456
```

首部字段	含义（常考）
Host	目的主机域名，HTTP/1.1 必需——一台服务器托多个站点靠它区分
Connection	`close` = 非持久（本次响应后就关 TCP）； `keep-alive` = 持久连接
Cookie	客户端回传的会话标识（HTTP 无状态，服务器靠它认出你是谁）

首部字段	含义（常考）
If-Modified-Since	条件 GET：对象未改则响应 **304**（无实体主体，用本地缓存）

响应报文 = 状态行（ `HTTP/1.1 200 OK` = 版本 + 状态码 + 短语）+ 首部行 + 空行 + 实体主体。常考响应首部： `Content-Length` （对象字节数）、 `Content-Type` （MIME 类型）、 `Last-Modified` （对象最后修改时间）、 `Set-Cookie` （服务器给客户端种 Cookie）。

常见坑

- WWW 是信息系统整体（URL+HTTP+HTML+海量超链接），HTTP 只是其中「传输」那一环；加载一个网页还会用到 DNS、TCP、TLS 等。
- 非持久连接每个对象 **2 RTT**（1 建连 + 1 请求/接收）；RTT 计数题里别漏最初取 HTML 的那 2 RTT。

  RTT 总时间、持久/非持久数值演算属计算，交给题（2022-40·2024-40）。展开博客精讲见 source_blogs。

FTP 协议 低频 对比表

- 基于 **TCP**，用**两个并行连接**（带外控制）

连接	端口	生命周期	用途
控制连接	**21**	整个会话期间保持	传 FTP 命令/响应
数据连接	**20**（主动模式）	每次传输建立、传完即关	传文件数据

- 主动模式 PORT：服务器从 **20 口主动连**客户端指定端口
- 被动模式 PASV：服务器开随机口，**客户端主动连**服务器（穿防火墙/NAT，更常用）
- 有状态协议（服务器维护当前目录/认证状态），对比 HTTP 无状态

怎么记不混

- **21 是控制、20 是数据**（主动模式）——最爱考反。被动模式的数据口是服务器**随机指定**的，不一定是 20。
- 「带外控制」= 命令和数据走不同连接（FTP）；HTTP 是「带内控制」，请求响应同一条 TCP 连接。

常见坑

- 主动模式下「服务器主动连客户端」在客户端处于防火墙/NAT 后会失败，所以实际多用被动模式（PASV，客户端主动连服务器）。
- FTP 有状态、HTTP 无状态，是常考辨析对。

  展开博客精讲见 source_blogs。

网络应用模型 冷门 对比表

维度	C/S（客户/服务器）	P2P（对等）
专用服务器	有，始终在线、固定 IP	无中心服务器
节点角色	服务器/客户分工固定	每个节点既是客户又是服务器
通信	客户只与服务器通信，**客户间不直接通信**	任意两节点可直接通信
可扩展性	差，受服务器性能限制	好，节点越多容量越大
健壮性	服务器故障即服务中断	单点故障影响小

- 典型：C/S → HTTP·FTP·SMTP；P2P → BitTorrent
- 文件分发：用户越多时 P2P 通常比 C/S 更快

怎么记不混

- **C/S 的「客户被动、客户间不通信」是 2019 年真题考点**：所有通信经服务器中转，两个浏览器之间不会直接交换数据。
- 「客户主动、服务器被动」只描述**建立连接**的过程；连接建好后服务器可以主动向客户推数据（推送/错误消息），别误判成「服务器永远不能主动发」。

常见坑

- P2P 实现上每次通信仍是一方当客户、一方当服务器——P2P 的区别在于**没有固定的服务器角色**，节点可灵活切换，不是「不存在客户/服务器机制」。

  展开博客精讲见 source_blogs。

扫码回站内背这套卡
站内按记忆曲线排期，到期自动进「今日待背」